Herhaling ARIMA

Kalman Filters

Tweede techniek voor tijdsreeksen: Kalman filter

- Als voorbeeldprobleem bekijken we een app, die de **GPS-locatie van een vliegtuig** aangeeft
- We proberen te **voorspellen** waar het vliegtuig naartoe gaat

```
flowchart LR
    G["GPS-metingen<br/>(met foutmarge)"] --> K["Kalman filter"]
    K --> S["✅ Glad traject<br/>+ voorspelling van positie"]
```

Het probleem met sensors

- Sensors, zoals GPS, bevatten een **foutmarge**
- Door die foutmarge kan het ruwe traject van de data er nogal 'bumpy' uitzien

```
flowchart TD
    R["Ruwe sensorwaardes"] --> B["Bumpy traject"]
    R --> O["Af en toe een afwijkende waarde"]
    O --> H["❌ Boordcomputer slaat op hol?"]
```

Kalman filter: predict en update

- De Kalman filter begint met een **initiële waarde**

```
flowchart LR
    I["Initiële waarde"] --> P["Predict-fase:<br/>nieuwe staat"]
    P --> C["(on)zekerheid over de staat"]
    C --> U["Update-fase:<br/>echte meting binnen"]
    U --> Q{"Staat 'dicht genoeg'<br/>bij de meting?"}
    Q --> P
```

Kalman filter: waarom nuttig?

- In de predict-fase: geeft een **nieuwe staat** en een **(on)zekerheid** over die staat
- In de update-fase (een echte meting): kijkt of de staat **'dicht genoeg'** is bij de meting
- Dit wordt in real life ook echt gebruikt in **boordnavigatie van vliegtuigen**
- De filter vermijdt het op hol slaan van een boordcomputer als 1 sensor een keer een **afwijkende waarde** meet

> 📌 Zie ook de oefening: `kalman_oefening_oplossing.ipynb`

Samenvatten

Kernpunten

- **Kalman filter** voor noisy metingen (predict + update)

Vragen?